Reporte

Nombre:	**Matrícula**
Ivan Francisco Padron Macias	2150830
María Ximena Rodarte Cuevas	2154111
Angel Haziel Bernal Garza	2156444

Nombre del curso: Componentes de lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Empresa:** Hotel Costa Azul
Fecha: 13/10/2026	
Bibliografia (Formato APA)	

Para este documento y la arquitectura del PMS del **Hotel Costa Azul**, el equipo definió e integró un total de 6 Registros de Decisiones Arquitectónicas (ADR). Estos registros justifican los patrones elegidos, la distribución de responsabilidades y las consecuencias asumidas para garantizar un sistema escalable, consistente y alineado con las reglas del negocio.

Resumen de las ADR

ADR	Decisión	Estado	Necesidades / Trazabilidad
ADR-001	Domain Model como patrón de lógica de negocio	Aceptada	RN-01 a RN-13, US-01 a US-15
ADR-002	Reserva y EstanciaFolio como Aggregate Roots; Habitación referenciada por ID	Aceptada	RN-01, RN-02, RN-04, RN-05, US-01, US-03, US-08
ADR-003	Domain Services para orquestar reglas complejas (TariffCalculationService y WalkService)	Aceptada	RN-03, RN-07, RN-09, US-02, US-04, US-09
ADR-004	Service Layer delgada, Repositorios por Agregado y Unit of Work	Aceptada	RN-05, RN-06, RN-12, US-01, US-08, US-13
ADR-005	CQRS básico con separación de Commands y Queries para la lectura de folios y disponibilidad	Aceptada	US-02, US-05, US-07, US-14, US-15
ADR-006	División de autorizaciones gerenciales entre Aplicación y Dominio	Aceptada	RN-08, RN-10, US-06, US-10, CA-007

ADR-001: Selección del patrón de lógica de negocio: Domain Model

Campo	Contenido
Título	Usar Domain Model para la gestión de reservas, folios y estancias del Hotel Costa Azul
Estado	Aceptada
Contexto	El PMS del Hotel Costa Azul maneja reglas críticas de negocio interrelacionadas (cálculo de tarifas dinámicas por temporada, control estricto de consumo en folios, garantía de no overbooking y políticas de cancelación/penalización). Un enfoque rudimentario podría provocar inconsistencias operativas en la recepción o pérdidas financieras.
Alternativas	• Transaction Script: Rápido de implementar inicialmente, pero dispersa las reglas de negocio en controladores/servicios, haciendo el sistema frágil y propenso a duplicar lógica de validación. • Table Module: Agrupa reglas por tabla de base de datos, pero acopla el diseño al esquema físico y dificulta modelar transiciones de estado complejas. • Domain Model: Encapsula reglas y datos en entidades e invariantes del dominio. Garantiza que la lógica viva en un solo lugar y se ejecute siempre antes de persistir.
Decisión	Adoptar Domain Model para el modelo de escritura (*write side*). Las entidades del dominio protegen sus propias invariantes. Las consultas de lectura simples (*read side*) se resolverán mediante proyecciones optimizadas en el esquema CQRS.
Consecuencias	(+) Las reglas clave del negocio hotelero se centralizan en el dominio y se pueden probar sin dependencia de base de datos. (+) El lenguaje ubicuo del Hotel Costa Azul se refleja directamente en el código. (-) Mayor complejidad inicial y número de clases respecto a un patrón CRUD simple.
Trazabilidad	RN-01 a RN-13, US-01 a US-15

ADR-002: Reserva y Folio como Aggregate Roots; Habitación referenciada por ID

Campo	Contenido
Título	Definición de límites transaccionales para Reserva y EstanciaFolio con referencias por ID
Estado	Aceptada
Contexto	Las operaciones de check-in, asignación de habitación, registro de consumo en folio y check-out tocan múltiples entidades dentro del hotel. Se requieren límites transaccionales claros para evitar bloqueos concurrentes en recepción.

Alternativas	• Un solo Agregado global del Hotel: Mantiene consistencia total pero causa cuellos de botella severos en concurrencia operativamente inaceptables. • Agregados independientes con referencias por ID + Unit of Work: Reserva y EstanciaFolio actúan como Aggregate Roots independientes. Habitación se referencia por su ID (HabitacionId).
Decisión	Establecer a Reserva y EstanciaFolio como los límites de agregados principales del sistema. La entidad Habitacion se referenciará únicamente por su identificador único dentro de los agregados.
Consecuencias	(+) Alta concurrencia y aislamiento transaccional por estancia o reserva individual. (+) Transacciones atómicas seguras: se persiste todo el cambio del agregado o nada. (-) Requiere coordinación entre agregados mediante eventos de dominio o servicios de aplicación.
Trazabilidad	RN-01, RN-02, RN-04, RN-05, US-01, US-03, US-08

ADR-003: Domain Services y bounded contexts: TariffCalculationService y WalkService

Campo	Contenido
Título	Implementación de Domain Services para cálculo de tarifas y gestión de overbooking
Estado	Aceptada
Contexto	El cálculo dinámico de tarifas (que considera tipo de habitación, temporada, promociones y garantía) y el protocolo de *walk* por overbooking (reubicación en hotel aliado) cruzan múltiples entidades y no pertenecen naturalmente a una sola entidad del dominio.
Alternativas	• Colocar la lógica en la Capa de Aplicación: Contamina los servicios de aplicación con reglas puras del negocio del hotel. • Recargar la entidad Reserva: Viola el principio de responsabilidad única al forzar a la reserva a conocer catálogos de tarifas globales u hoteles externos • Crear Domain Services especializados: Clases sin estado en el Dominio que encapsulan la orquestación de la regla.
Decisión	Crear los servicios de dominio TariffCalculationService (para la cotización y tarifación) y WalkService (para gestionar la política de reubicación y compensación por overbooking).
Consecuencias	(+) Mantiene los agregados limpios y enfocados en sus invariantes. (+) Reutilización sencilla del cálculo tarifario en cotización y reserva. (-) Se requiere control estricto para evitar que los Domain Services le quiten la lógica propia a las entidades.
Trazabilidad	RN-03, RN-07, RN-09, US-02, US-04, US-09

ADR-004: Service Layer delgada, Repositories por agregado y Unit of Work

Campo	Contenido
Título	Capa de aplicación delgada con repositorios abstractos en el dominio
Estado	Aceptada
Contexto	Se necesita desacoplar los casos de uso (orquestación) de los detalles de infraestructura de persistencia para facilitar pruebas automatizadas e independencia tecnológica en el PMS.
Alternativas	• Servicios de aplicación con llamadas directas a base de datos: Acopla fuertemente el PMS al framework ORM o motor de BD. • Repositorios abstractos en el Dominio e implementación en Infraestructura: La capa de aplicación interactúa solo con interfaces (IReservaRepository, IFolioRepository).
Decisión	Mantener una Service Layer delgada (solo orquesta DTOs, repositorios y transacciones). Las interfaces de repositorio residen en el Dominio y se implementan mediante el patrón Unit of Work en la capa de Infraestructura.
Consecuencias	(+) Desacoplamiento total del motor de base de datos; pruebas unitarias aisladas. (+) Cumplimiento estricto con el principio de Inversión de Dependencias (DIP). (-) Aumento en el número de interfaces y mapeadores entre DTOs, entidades y objetos de BD.
Trazabilidad	RN-05, RN-06, RN-12, US-01, US-08, US-13

ADR-005: CQRS básico con Commands, Queries y DTOs de lectura

Campo	**Contenido**
Título	Separación entre operaciones de modificación (Commands) y lecturas del PMS (Queries)
Estado	Aceptada
Contexto	La consulta de disponibilidad de habitaciones o la búsqueda de folios por parte de recepción requiere rapidez y no necesita validar reglas de negocio complejas ni instanciar agregados pesados, a diferencia de las operaciones de modificación (crear reserva, realizar cobro).
Alternativas	• Cargar el modelo de dominio completo para lecturas: Causa penalizaciones innecesarias de rendimiento al instanciar agregados completos solo para mostrar datos en pantalla.CQRS

	• Básico: Dividir la aplicación en Commands (pasan por las entidades del dominio para modificar estado) y Queries (consultan proyecciones/DTOs ligeros directamente).
Decisión	Adoptar CQRS básico. Los Commands invocan agregados e instancian la lógica del dominio; las Queries retornan DTOs de lectura optimizados para la interfaz de usuario.
Consecuencias	(+) Respuesta inmediata en pantallas operativas de recepción (disponibilidad y búsqueda de folios). (+) El modelo de dominio permanece enfocado exclusivamente en consistencia transaccional. (-) Mantenimiento de DTOs y modelos de lectura independientes de las entidades de dominio.
Trazabilidad	US-02, US-05, US-07, US-14, US-15

ADR-006: Autorización gerencial dividida entre aplicación y dominio

Campo	Contenido
Título	Manejo de autorizaciones, descuentos y anulaciones entre Aplicación y Dominio
Estado	Aceptada
Contexto	Reglas operativas como anulaciones de consumo, descuentos especiales o exoneraciones requieren aprobación explícita de un gerente del Hotel Costa Azul (RN-08, RN-10). Se requiere normar dónde se valida dicho privilegio.
Alternativas	**Validar permisos dentro del Dominio:** Contamina el dominio con conceptos de seguridad, tokens, roles e infraestructura de usuario. **Validar permisos únicamente en la UI:** Riesgo grave de seguridad si el servicio se consume sin pasar por la vista. **Modelo Mixto:** La Capa de Aplicación valida las credenciales/rol gerencial y pasa un Value Object de autorización al Dominio.
Decisión	La **Capa de Aplicación** autentica y valida las credenciales gerenciales. Si es válido, pasa el objeto AutorizacionGerencial al método del **Dominio**, el cual exige este comprobante para autorizar el descuento o anulación.
Consecuencias	**(+)** Dominio limpio sin acoplamiento a frameworks de seguridad o autenticación. **(+)** Garantiza que ninguna acción restringida del hotel se aplique sin la evidencia del objeto de autorización válido.

	(-) Se deben pasar parámetros de autorización de forma explícita a los métodos del dominio.
Trazabilidad	RN-08, RN-10, US-06, US-10, CA-007